

پیاده‌سازی الگوریتم ژنتیک برای انتخاب ویژگی‌های مؤثر در طبقه‌بندی
مجموعه داده Iris

نام درس: داده‌کاوی

نام استاد: دکتر شفیعی

نام اعضای گروه: الناز رادمهر ۴۰۱۱۴۰۰۴

رشته: علوم کامپیوتر

دانشگاه: صنعتی کرمانشاه

تاریخ: ۱۴۰۵/۰۶/۱۳

۱. مقدمه

هدف این پروژه، آشنایی عملی با نحوه عملکرد الگوریتم ژنتیک و اجرای آن روی یک مجموعه داده واقعی است. برای انجام این کار، مجموعه داده Iris را انتخاب کردم و الگوریتم ژنتیک را برای انتخاب ویژگی‌های مناسب آن پیاده‌سازی کردم.

در این پروژه از الگوریتم **KNN** برای طبقه‌بندی داده‌ها استفاده کردم و الگوریتم ژنتیک را به عنوان روشی برای انتخاب ویژگی به کار بردم. به این صورت که هر جواب الگوریتم ژنتیک مشخص می‌کند از بین ویژگی‌های موجود، کدام ویژگی‌ها در مدل **KNN** استفاده شوند.

در ادامه ابتدا داده‌ها را آماده کردم و عملکرد **KNN** را با استفاده از تمام ویژگی‌ها به عنوان مدل پایه به دست آوردم. سپس الگوریتم ژنتیک را طراحی و اجرا کردم تا ترکیب‌های مختلف ویژگی‌ها را بررسی کند. در پایان، بهترین ترکیب ویژگی‌ها را روی داده‌های آزمون ارزیابی کردم.

۲. تعریف مسئله

مسئله مورد بررسی در این پروژه، **انتخاب ویژگی (Feature Selection)** برای طبقه‌بندی مجموعه داده Iris است.

مجموعه داده Iris دارای چهار ویژگی است:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

هدف من این بود که با استفاده از الگوریتم ژنتیک، ترکیبی از این ویژگی‌ها را پیدا کنم که در طبقه‌بندی عملکرد مناسبی داشته باشد و در صورت امکان تعداد ویژگی‌های مورد استفاده نیز کاهش پیدا کند.

برای نمایش هر جواب از یک کروموزوم دودویی استفاده کردم. در این نمایش، مقدار هر ژن مشخص می‌کند که ویژگی مربوطه انتخاب شده است یا خیر:

۱: ویژگی انتخاب شده

۰: ویژگی انتخاب نشده

برای مثال، کروموزوم زیر:

[۱ ۱ ۰ ۱]

یعنی ویژگی‌های اول، دوم و چهارم انتخاب شده‌اند و ویژگی سوم استفاده نشده است.

بنابراین مسئله اصلی الگوریتم ژنتیک در این پروژه، پیدا کردن بهترین ترکیب صفر و یک برای چهار ویژگی موجود است.

۳. اعضای گروه

این پروژه به صورت انفرادی انجام شده است.

تمام مراحل پروژه توسط خودم انجام شده است؛ از جمله انتخاب دیتاست، آماده‌سازی داده‌ها، پیاده‌سازی الگوریتم ژنتیک، تعریف تابع Fitness، پیاده‌سازی Selection، Crossover و Mutation، اجرای برنامه، بررسی خروجی‌ها، تحلیل نتایج و تهیه گزارش.

نکته مهم: در تمام مراحل از هوش مصنوعی استفاده کردم، چون زمان ترم کوتاه بود و برای پیاده‌سازی کدها به کمک بیشتری نیاز داشتم. با این حال، همه کدها را خطبه‌خط بررسی و یاد گرفتم و با مفهوم و روند اجرای الگوریتم به خوبی آشنا شدم. امیدوارم مورد قبول واقع شود.

۴. معرفی مجموعه داده Iris

برای اجرای الگوریتم، مجموعه داده Iris را انتخاب کردم. این مجموعه داده یکی از دیتاست‌های شناخته‌شده در یادگیری ماشین است و شامل اطلاعات مربوط به سه نوع گل زنبق است.

این مجموعه داده شامل ۱۵۰ نمونه و ۴ ویژگی است.

مقدار مشخصات

تعداد
۱۵۰
نمونه‌ها

تعداد
۴
ویژگی‌ها

تعداد
۳
کلاس‌ها

چهار ویژگی موجود در دیتاست عبارتند از:

۱. Sepal Length

۲. Sepal Width

۳. Petal Length

۴. Petal Width

برای دریافت داده از کتابخانه `scikit-learn` استفاده کردم:

```
from sklearn.datasets import load_iris
iris = load_iris()
x = iris.data
y = iris.target
```

سپس با استفاده از کد زیر ابعاد دیتاست را بررسی کردم:

```
print("Dataset shape: ", x.shape)
```

خروجی:

```
Dataset shape: (150, 4)
```

این خروجی نشان می‌دهد که دیتاست شامل ۱۵۰ نمونه و ۴ ویژگی است.

```
Dataset shape: (150, 4)
Training shape: (120, 4)
Testing shape: (30, 4)
Baseline accuracy: 1.0
```

۵. تقسیم داده‌ها

بعد از دریافت دیتاست، داده‌ها را به دو بخش آموزش و آزمون تقسیم کردم. در این پروژه ۸۰ درصد داده‌ها برای آموزش و ۲۰ درصد برای آزمون در نظر گرفته شد.

```
x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

در نتیجه ۱۲۰ نمونه برای آموزش و ۳۰ نمونه برای آزمون در نظر گرفته شد.

خروجی برنامه:

Training shape: (120, 4)

Testing shape: (30, 4)

برای اینکه الگوریتم ژنتیک بتواند ویژگی‌ها را انتخاب کند، بخشی از داده‌های آموزشی را نیز به دو قسمت Training و Validation تقسیم کردم.

دلیل استفاده از Validation این است که الگوریتم ژنتیک هنگام انتخاب ویژگی‌ها از داده Test استفاده نکند. بنابراین داده Test تا مرحله ارزیابی نهایی کنار گذاشته شد.

۶. ایجاد مدل پایه KNN

قبل از اجرای الگوریتم ژنتیک، ابتدا یک مدل KNN با استفاده از هر چهار ویژگی ایجاد کردم.

هدف از این مرحله این بود که یک معیار اولیه برای مقایسه داشته باشم و بعد ببینم انتخاب ویژگی با الگوریتم ژنتیک چه تغییری در عملکرد مدل ایجاد می‌کند.

کد مربوط به این مرحله:

```
model = KNeighborsClassifier(n_neighbors=5)
```

```
model.fit(x_train, y_train)
```

```
y_pred = model.predict(x_test)
```

```
baseline_accuracy = accuracy_score(y_test, y_pred)
```

در اینجا مقدار K برابر ۵ انتخاب شده است.

خروجی:

Baseline accuracy: 1.0

بنابراین مدل پایه KNN با استفاده از تمام چهار ویژگی، روی داده Test به دقت ۱۰۰ درصد رسید.

۷. نمایش ویژگی‌ها به صورت کروموزوم

در مرحله بعد باید مشخص می‌کردم که الگوریتم ژنتیک چگونه ویژگی‌ها را نمایش دهد.

چون دیتاست چهار ویژگی دارد، طول هر کروموزوم را برابر با ۴ قرار دادم.

برای هر ویژگی یک ژن در نظر گرفتم:

[Feature 1, Feature 2, Feature 3, Feature 4]

مقدار هر ژن نیز صفر یا یک است.

برای مثال:

[۱ ۰ ۱ ۱]

یعنی ویژگی‌های اول، سوم و چهارم انتخاب شده‌اند.

این نوع نمایش باعث می‌شود بررسی ترکیب‌های مختلف ویژگی‌ها برای الگوریتم ژنتیک ساده باشد.

۸. ایجاد جمعیت اولیه

بعد از مشخص کردن ساختار کروموزوم، جمعیت اولیه را ایجاد کردم.

در این پروژه اندازه جمعیت برابر ۲۰ در نظر گرفته شد:

```
population_size = 20
```

```
number_of_features = x.shape[1]
```

```
population = np.random.randint(
```

```
0,
```

```
2,
```

```
size=(population_size, number_of_features)
```

```
)
```

بنابراین در شروع الگوریتم، ۲۰ کروموزوم تصادفی ایجاد می‌شود و هر کروموزوم چهار ژن دارد.

نمونه‌ای از جمعیت ایجادشده:

```
Initial population:
[[1 0 1 1]
 [1 1 1 0]
 [1 1 0 0]
 [1 0 0 1]
 [0 1 1 1]
 [0 1 1 0]
 [1 0 0 1]
 [1 1 0 1]
 [1 1 1 0]
 [0 0 0 0]
 [0 1 0 1]
 [0 0 1 0]]
```

در این مرحله هنوز مشخص نیست کدام کروموزوم بهترین جواب است، بنابراین در مرحله بعد Fitness هر کروموزوم محاسبه می‌شود.

۹. محاسبه Fitness

برای مقایسه کروموزوم‌ها باید مشخص شود که هر کدام چقدر مناسب هستند. برای این کار تابع Fitness را تعریف کردم.

ابتدا مشخص می‌شود کدام ویژگی‌ها توسط کروموزوم انتخاب شده‌اند:

```
selected_features = chromosome == 1
```

سپس فقط همان ویژگی‌ها از داده‌های آموزشی و Validation جدا می‌شوند:

```
x_train_selected = x_train_ga[:, selected_features]
```

```
x_validation_selected = x_validation[:, selected_features]
```

در مرحله بعد KNN با ویژگی‌های انتخاب‌شده آموزش داده می‌شود و دقت آن روی Validation محاسبه می‌شود.

برای اینکه الگوریتم فقط به دنبال دقت بالا نباشد و در صورت امکان تعداد ویژگی‌ها را نیز کاهش دهد، یک جریمه کوچک برای تعداد ویژگی‌های انتخاب‌شده در نظر گرفتیم.

فرمول Fitness به صورت زیر است:

Fitness = Accuracy – Feature Penalty

و جریمه به شکل زیر محاسبه می‌شود:

```
feature_penalty = 0.01 * (
```

```
number_of_selected_feature / len(chromosome))
```

)

$$\text{fitness} = \text{accuracy} - \text{feature_penalty}$$

بنابراین هرچه دقت بیشتر و تعداد ویژگی‌های انتخاب شده کمتر باشد، Fitness مناسب‌تر خواهد بود.

۱۰. انتخاب کروموزوم‌ها (Selection)

بعد از محاسبه Fitness، باید مشخص شود کدام کروموزوم‌ها برای تولید نسل بعد انتخاب شوند.

در این پروژه از روش **Tournament Selection** استفاده کردم.

در این روش دو کروموزوم به صورت تصادفی انتخاب می‌شوند و Fitness آن‌ها با یکدیگر مقایسه می‌شود. کروموزومی که Fitness بیشتری داشته باشد، برنده می‌شود و برای تولید نسل بعد انتخاب خواهد شد.

قسمت اصلی کد:

```
if fitness_value[first] > fitness_value[second]:
```

```
    winner = first
```

```
else:
```

```
    winner = second
```

با این روش، کروموزوم‌های بهتر شانس بیشتری برای انتقال به نسل بعد پیدا می‌کنند.

۱۱. عملگر Crossover

بعد از انتخاب والدین، مرحله Crossover انجام می‌شود.

در این مرحله یک نقطه تصادفی در کروموزوم انتخاب می‌شود و قسمت‌هایی از دو والد با یکدیگر ترکیب می‌شوند.

برای مثال:

Parent 1: [1 1 | 1 0]

Parent 2: [0 1 | 0 1]

ممکن است یکی از فرزندان به صورت زیر ایجاد شود:

Child: [1 1 | 0 1]

هدف Crossover این است که ویژگی‌های موجود در والدین مختلف با یکدیگر ترکیب شوند و جواب‌های جدید ایجاد شوند.

۱۲. عملگر Mutation

بعد از Crossover، عملگر Mutation را اجرا کردم.

در این پروژه نرخ Mutation برابر با ۰.۱ است.

در صورت رخ دادن Mutation، مقدار ژن تغییر می‌کند.

برای مثال:

قبل:

[۱ ۱ ۰ ۱]

بعد:

[۱ ۰ ۰ ۱]

در این مثال مقدار ژن دوم از ۱ به ۰ تغییر کرده است.

Mutation باعث می‌شود الگوریتم امکان بررسی ترکیب‌های جدید را داشته باشد.

۱۳. ایجاد نسل جدید و حفظ بهترین جواب

بعد از Selection، Crossover و Mutation، جمعیت جدید ساخته می‌شود.

در این پروژه بهترین کروموزوم هر نسل مستقیماً به نسل بعد منتقل می‌شود. این روش **Elitism** نام دارد.

```
new_population = []
```

```
new_population.append(best_chromosome.copy())
```

با این کار اگر الگوریتم در یک نسل به جواب خوبی برسد، آن جواب به دلیل Mutation یا Crossover از بین نمی‌رود.

۱۴. اجرای الگوریتم ژنتیک

پس از پیاده‌سازی تمام قسمت‌های الگوریتم، آن را برای ۳۰ نسل اجرا کردم.

در هر نسل ابتدا Fitness تمام کروموزوم‌ها محاسبه می‌شود. سپس بهترین کروموزوم مشخص شده و با استفاده از Selection،

Crossover و Mutation جمعیت جدید ایجاد می‌شود.

روند کلی اجرا به صورت زیر است:

Population اولیه



Fitness محاسبه



Selection



Crossover



Mutation



حفظ بهترین کروموزوم



ایجاد نسل جدید



تکرار تا نسل ۳۰

در هر نسل مقدار بهترین Fitness نیز در خروجی برنامه نمایش داده شد.

۱۵. بهترین جواب به دست آمده

پس از اجرای ۳۰ نسل، بهترین جواب کلی الگوریتم به صورت زیر به دست آمد:

Best Overall Solution

Best chromosome: [1 1 1 0]

Best fitness: 0.9925

کروموزوم به دست آمده:

[۱ ۱ ۱ ۰]

به این معنی است که سه ویژگی اول انتخاب شده‌اند و ویژگی چهارم انتخاب نشده است.

بنابراین ویژگی‌های انتخاب شده عبارت‌اند از:

• Sepal Length

• Sepal Width

• Petal Length

و ویژگی چهارم یعنی:

• Petal Width

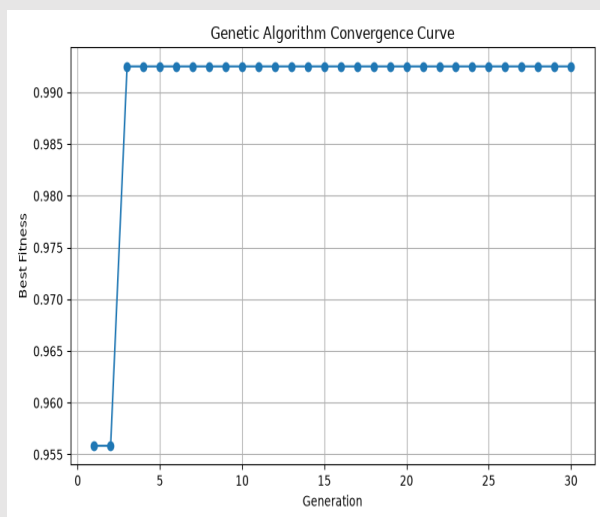
در این جواب استفاده نشده است.

```
Best Overall Solution
-----
Best chromosome: [1 1 1 0]
Best fitness: 0.9925
```

نکته مهم این است که مقدار ۰,۹۹۲۵ مربوط به **Fitness** است و نه **Accuracy**. در این پروژه **Fitness** علاوه بر دقت، تعداد ویژگی‌های انتخاب شده را نیز در نظر می‌گیرد.

۱۶. نمودار همگرایی الگوریتم

برای بررسی روند تغییر بهترین **Fitness** در نسل‌های مختلف، مقدار **Best Fitness** هر نسل را ذخیره کردم و با استفاده از کتابخانه **Matplotlib** نمودار آن را رسم کردم.



نمودار نشان می‌دهد که الگوریتم در نسل‌های ابتدایی به یک جواب مناسب رسیده و پس از چند نسل مقدار بهترین **Fitness** تقریباً ثابت شده است.

این موضوع نشان می‌دهد که در اجرای انجام شده، الگوریتم نسبتاً سریع به بهترین جواب پیداشده رسیده و در نسل‌های بعدی بهبود قابل توجهی ایجاد نشده است.

۱۷. ارزیابی نهایی

بعد از پیدا شدن بهترین کروموزوم، ویژگی‌های مشخص شده توسط آن را انتخاب کردم و یک مدل KNN جدید با همین ویژگی‌ها روی داده‌های آموزشی آموزش دادم.

سپس مدل روی داده Test ارزیابی شد.

```
Final Evaluation
-----
Selected features: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)']
Number of selected features: 3
Final test accuracy: 0.9333
```

بنابراین مدل نهایی با سه ویژگی انتخاب شده توانست به دقت ۹۳,۳۳ درصد روی مجموعه Test برسد.

۱۸. مقایسه با مدل پایه

در پایان، نتیجه مدل پایه و مدل دارای ویژگی‌های انتخاب شده را با یکدیگر مقایسه کردم.

جدول نتایج:

مدل	تعداد	دقت روی Test
KNN با تمام ویژگی‌ها	۴	۱۰۰٪
KNN با ویژگی‌های انتخاب شده توسط GA	۳	۹۳.۳۳٪

همان‌طور که مشاهده می‌شود، الگوریتم ژنتیک تعداد ویژگی‌ها را از ۴ به ۳ کاهش داده است. بنابراین ۲۵ درصد کاهش در تعداد ویژگی‌ها اتفاق افتاده است.

در مقابل، دقت مدل از ۱۰۰ درصد به ۹۳,۳۳ درصد کاهش پیدا کرده است.

۱۹. تحلیل نتیجه

نتیجه این اجرا نشان می‌دهد که الگوریتم ژنتیک توانسته است از بین ترکیب‌های مختلف ویژگی‌ها، یک ترکیب سه‌تایی را پیدا کند که عملکرد قابل‌قبولی داشته باشد.

البته در این آزمایش، کاهش تعداد ویژگی‌ها باعث افزایش دقت نشده است و مدل پایه که از هر چهار ویژگی استفاده می‌کند، دقت بیشتری روی داده Test داشته است.

این نتیجه لزوماً به معنی نامناسب بودن الگوریتم ژنتیک نیست؛ زیرا هدف این پروژه بررسی فرایند انتخاب ویژگی بود. در تابع Fitness علاوه بر دقت، تعداد ویژگی‌های انتخاب‌شده نیز در نظر گرفته شده است. بنابراین الگوریتم سعی می‌کند بین عملکرد مدل و تعداد ویژگی‌ها یک تعادل ایجاد کند.

همچنین الگوریتم ژنتیک برای انتخاب ویژگی‌ها از داده Validation استفاده کرده و داده Test فقط در مرحله نهایی استفاده شده است. به همین دلیل مقدار Fitness و دقت نهایی Test یکسان نیستند.

۲۰. نتیجه‌گیری

در این پروژه الگوریتم ژنتیک را برای انتخاب ویژگی‌های مناسب در مجموعه داده Iris پیاده‌سازی و اجرا کردم.

ابتدا داده‌های Iris را دریافت و به مجموعه‌های آموزش، Validation و Test تقسیم کردم. سپس یک مدل KNN با تمام ویژگی‌ها ایجاد کردم که به عنوان مدل پایه، دقت ۱۰۰ درصد را روی داده Test به دست آورد.

در مرحله بعد، هر ترکیب از ویژگی‌ها را به صورت یک کروموزوم دودویی نمایش دادم و یک جمعیت اولیه شامل ۲۰ کروموزوم ایجاد کردم. برای ارزیابی هر کروموزوم، دقت KNN روی داده Validation را محاسبه کردم و یک جریمه کوچک نیز برای تعداد ویژگی‌های انتخاب‌شده در نظر گرفتم.

سپس با استفاده از Selection، Crossover و Mutation نسل‌های جدید را ایجاد کردم و بهترین جواب هر نسل را نیز حفظ کردم. الگوریتم برای ۳۰ نسل اجرا شد.

در پایان، بهترین کروموزوم به صورت:

[۰ ۱ ۱ ۱]

به دست آمد. این کروموزوم سه ویژگی از چهار ویژگی موجود را انتخاب کرد و مقدار Fitness آن برابر با ۰.۹۹۲۵ بود.

در ارزیابی نهایی، مدل KNN با سه ویژگی انتخاب‌شده به دقت ۹۳.۳۳ درصد رسید. بنابراین تعداد ویژگی‌ها ۲۵ درصد کاهش پیدا کرد، اما نسبت به مدل پایه کاهش دقت نیز مشاهده شد.

این پروژه برای من نشان داد که الگوریتم ژنتیک می‌تواند برای جست‌وجو در میان ترکیب‌های مختلف ویژگی‌ها استفاده شود و مراحل اصلی آن، شامل ایجاد جمعیت، ارزیابی Fitness، انتخاب، Crossover، Mutation و حفظ بهترین جواب است.

۲۱. کتابخانه‌ها و ابزارهای استفاده‌شده

در این پروژه از زبان برنامه‌نویسی **Python** و کتابخانه‌های زیر استفاده کردم:

- **NumPy**: برای ایجاد و مدیریت جمعیت و کروموزوم‌ها
- **Scikit-learn**: Accuracy و محاسبه KNN، تقسیم داده‌ها، الگوریتم Iris برای دیتاست
- **Matplotlib**: برای رسم نمودار همگرایی الگوریتم

۲۲. لینک پروژه در GitHub

کدهای پیاده‌سازی الگوریتم ژنتیک، فایل‌های موردنیاز پروژه و مستندات مربوط به آن در مخزن GitHub پروژه قرار داده شده است.

GitHub Repository:

<https://github.com/elnazradmehr/genetic-algorithm-iris>